

DOCKER

Detailed Notes - Beginner to Advanced

A practical study guide covering Docker fundamentals, images, containers, Dockerfiles, networking, storage, Compose, registries, security, debugging, optimization, and interview preparation.

Core idea: Docker packages an application and its dependencies into a reproducible image and runs that image as an isolated container.

1. Docker Fundamentals

What is Docker?

Docker is a platform for building, shipping, and running applications in lightweight isolated environments called containers. Containers share the host operating system kernel while isolating processes, filesystems, networking, and resources.

Why use Docker?

It reduces environment mismatch, makes development and CI/CD more repeatable, simplifies dependency management, speeds application startup, and makes services easier to package and deploy.

Docker vs VMs

A virtual machine normally contains a complete guest OS. A container shares the host kernel and is usually lighter and faster to start. VMs can provide stronger OS-level isolation; containers are highly useful for application packaging.

2. Docker Architecture

Main pieces

Docker CLI: sends commands. **Docker Engine:** manages Docker objects. **Images:** read-only templates. **Containers:** executable image instances. **Registries:** store and distribute images. **Volumes:** persistent data. **Networks:** service connectivity.

Mental model

Dockerfile -> **docker build** -> **Image** -> **docker run** -> **Container**. An image can be pushed to a registry and pulled by another host.

3. Essential Commands

Verify

```
docker --version
docker info
docker run hello-world
```

Images

```
docker images - list images
docker pull nginx - download image
docker image inspect nginx - inspect metadata
docker history nginx - show layers/history
docker rmi IMAGE - remove image
```

Containers

```
docker ps - running containers
docker ps -a - all containers
docker run IMAGE - create/start
docker start NAME - start stopped container
docker stop NAME - stop
docker restart NAME - restart
docker rm NAME - remove
```

Interactive and detached

`docker run -it ubuntu bash` opens a terminal. `docker run -d nginx` runs in background.

Name and publish ports

`docker run --name web -d -p 8080:80 nginx`. Syntax is **HOST_PORT:CONTAINER_PORT**. **EXPOSE** does not publish a host port.

4. Images and Layers

Image definition

An image is a layered, read-only package containing application files plus metadata such as startup command and environment configuration.

Tags and digests

A tag such as `1.0` is a readable reference. A digest identifies exact image content. Version tags or digests are preferable to relying on a moving tag such as `latest` for reproducible deployments.

Layers

Image layers represent filesystem changes. Reusing unchanged layers makes builds faster and reduces duplicate storage.

5. Dockerfile in Detail

What is it?

A Dockerfile is a text recipe used by `docker build` to create an image.

Important instructions

FROM base image. **WORKDIR** working directory. **COPY** copy files. **ADD** has extra behavior; **COPY** is preferred for ordinary copying. **RUN** executes during build. **ENV** configures environment. **ARG** build-time variable. **EXPOSE** documents a port. **CMD** default command/arguments. **ENTRYPOINT** executable. **USER** runtime user. **HEALTHCHECK** health test.

Node.js example

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Build

`docker build -t myapp:1.0 .` The final dot is the build context.

Good practices

Use a small trusted base image, install only required packages, avoid secrets, use `.dockerignore`, use non-root users where practical, and structure instructions to benefit from cache reuse.

6. Build Context, Cache, and `.dockerignore`

Build context

The build context is the set of files available to the build. Large contexts slow builds and can accidentally include credentials or local dependency directories.

Cache-friendly order

For Node.js, a common pattern is: **COPY** package files -> **RUN** `npm ci` -> **COPY** source. If only source code changes, the dependency layer can often stay cached.

.dockerignore

Typical entries include `node_modules`, `.git`, `logs`, local build output, editor folders, and environment files that should not enter the build context.

7. CMD vs ENTRYPOINT

CMD

CMD specifies default command or default arguments. It is easy to override when starting the container.

ENTRYPOINT

ENTRYPOINT makes the image behave like a specific executable; CMD can provide default arguments to it.

Rule of thumb

Use ENTRYPOINT when the container is designed around one main executable interface; use CMD for replaceable defaults.

8. Environment Variables and Secrets

Environment

`docker run -e NODE_ENV=production -e PORT=3000 myapp` injects runtime configuration.

ARG vs ENV

ARG is primarily a build-time value. ENV becomes image/container environment configuration.

Secret safety

Never hard-code API keys, passwords, or tokens into Dockerfiles or image layers. Use deployment-appropriate secret management.

9. Volumes and Storage

Why persist data?

The container writable layer is not a durable database strategy. Important state should normally live in volumes or external storage.

Named volume

```
docker volume create mydata
docker run -d --mount source=mydata,target=/data redis
```

Bind mount

`docker run -v "$PWD":/app node:22`. Useful for local development and direct host-file access.

Commands

```
docker volume ls, docker volume inspect NAME, docker volume rm NAME
```

10. Docker Networking

Purpose

Networks let containers communicate with each other and with external systems.

Common drivers

bridge for common single-host networking; **host** uses the host network stack; **none** disables normal networking; **overlay** can connect Docker nodes in appropriate multi-node setups.

User-defined bridge

```
docker network create appnet
docker run -d --name db --network appnet postgres:17
docker run -d --name api --network appnet myapi
```

Service discovery

On a user-defined network, containers can commonly communicate using container names rather than changing IP addresses.

11. Docker Compose

Why Compose?

Compose defines a multi-container application in YAML: services, networks, volumes, ports, environment, and build settings. It is excellent for development and repeatable local environments.

Example

```
services:
  web:
    build: .
    ports:
      - "3000:3000"
    depends_on:
      - db
  db:
    image: postgres:17
    environment:
      POSTGRES_PASSWORD: example
    volumes:
      - dbdata:/var/lib/postgresql/data
volumes:
  dbdata:
```

Commands

```
docker compose up
docker compose up -d
docker compose down
docker compose ps
docker compose logs -f
docker compose build
docker compose exec web sh
```

depends_on warning

Startup order is not the same as application readiness. Health checks and retry logic may still be required.

12. Registries and Docker Hub

Registry role

A registry stores and distributes images. Public and private registries can be used.

Push flow

```
docker login
docker tag myapp:1.0 username/myapp:1.0
docker push username/myapp:1.0
docker pull username/myapp:1.0
```

13. Multi-Stage Builds

Concept

Use multiple FROM stages so build dependencies stay in an intermediate stage and the final image contains only runtime files.

Example

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build
```

```
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
```

Benefits

Smaller runtime image, fewer unnecessary tools, smaller attack surface, and cleaner build/runtime separation.

14. Health Checks

Purpose

Health checks test whether the application appears healthy, not merely whether a process exists.

Example

```
HEALTHCHECK --interval=30s --timeout=5s --retries=3 CMD wget -qO- http://localhost:3000/health || exit 1
```

15. Debugging and Logs

Basic sequence

Check `docker ps -a`, then `docker logs CONTAINER`, then `docker inspect CONTAINER`. Use `docker stats` for resources.

Enter container

```
docker exec -it CONTAINER sh
```

Container exits immediately

Check logs, CMD/ENTRYPOINT, required environment variables, and remember the container normally stays alive while its main process is running.

Port problems

Verify the app listen address/port, host-to-container port mapping, firewall/proxy rules, and Docker network membership.

16. Security

Least privilege

Avoid root when unnecessary. Use USER, drop unnecessary capabilities, avoid privileged mode, and restrict access to the Docker daemon/socket.

Images

Use trusted base images, scan images and dependencies, remove unnecessary packages, keep versions updated, and never bake secrets into images.

Runtime

Use read-only filesystems where possible, limit CPU/memory, mount only required host paths, and expose only required ports.

Dangerous option

--privileged grants extensive additional privileges and should not be used casually.

17. Resource Limits and Monitoring

Limits

`docker run --memory=512m --cpus=1.0 myapp` restricts memory and CPU usage.

Monitoring

`docker stats` provides live resource usage. Combine container metrics with application logs and health checks.

18. Cleanup

Prune

`docker container prune` removes stopped containers. `docker image prune` removes dangling images. `docker system prune` performs broader cleanup; review deletions before using it on important systems.

19. Docker Workflow for a Real Project

Typical workflow

1. Write application code.
2. Create Dockerfile.
3. Create .dockerignore.
4. Build image with version tag.
5. Run locally with environment variables, ports, and volumes.
6. Add Compose if multiple services are needed.
7. Test and inspect logs/health.
8. Push image to registry.
9. Deploy versioned image.
10. Monitor and roll forward/back when required.

20. Common Interview Questions

Image vs container

Image is the read-only template; container is a runnable instance.

EXPOSE vs -p

EXPOSE documents a container port; -p publishes/maps a host port.

COPY vs ADD

COPY is the simple copy instruction. ADD has additional behavior and is less often necessary.

Volume vs bind mount

Volumes are Docker-managed; bind mounts map host paths directly.

Why a container stops

Its main process exits or crashes.

Why Compose

To define and operate related services together.

Why multi-stage

To keep build dependencies out of runtime and reduce image size.

Registry

A service that stores and distributes container images.

Layer

A filesystem change stored as part of an image layer stack, enabling reuse and caching.

21. Mini Project: Node API + PostgreSQL

Architecture

Run a Node.js API in one container and PostgreSQL in another. Connect them through a Compose network and persist database data in a named volume.

Files

```
project/  
■■■ Dockerfile  
■■■ compose.yaml  
■■■ .dockerignore  
■■■ package.json  
■■■ src/
```

Start

```
docker compose up --build  
docker compose ps  
docker compose logs -f
```

22. Best Practices

Golden rules

Keep images small and reproducible. Use .dockerignore. Prefer multi-stage builds where useful. Do not bake secrets into images. Run with least privilege. Persist important data outside ephemeral layers. Publish only required ports. Add health checks. Monitor logs and resources. Use explicit image versions for controlled deployments.

23. Command Cheat Sheet

Images

```
docker images | docker pull IMAGE | docker build -t NAME:TAG . | docker rmi IMAGE | docker  
history IMAGE
```

Containers

`docker run|docker ps|docker ps -a|docker start|docker stop|docker rm|docker exec -it CONTAINER sh`

Debug

`docker logs|docker inspect|docker stats|docker top`

Network

`docker network ls|docker network create NAME|docker network inspect NAME`

Volume

`docker volume ls|docker volume create NAME|docker volume inspect NAME`

Compose

`docker compose up|docker compose down|docker compose ps|docker compose logs -f|docker compose exec SERVICE sh`

24. Learning Roadmap

Progression

Stage 1: commands and containers. Stage 2: Dockerfile and images. Stage 3: ports, networks, volumes. Stage 4: Compose. Stage 5: debugging and security. Stage 6: multi-stage builds and CI/CD. Stage 7: containerize your own project and deploy it.

Memory trick

Dockerfile builds an image. Image creates a container. Container runs the application. Network connects services. Volume persists data. Compose coordinates services. Registry distributes images.

Final Revision Map

Code -> Dockerfile -> Build -> Image -> Run -> Container -> Ports/Networks -> Volumes -> Logs/Health -> Registry -> Deployment

Interview-ready definition

Docker is a containerization platform that packages software and its dependencies into portable images and runs those images as isolated containers, making application environments more consistent across development, testing, and deployment.